

PRODUCT REQUIREMENTS DOCUMENT

ChildTaskTracker

A shared, AI-assisted chore & routine tracker for kids ages 6–16 — child self-reports, a parent validates, and lightweight AI turns the data into kid-friendly progress and reward suggestions.

STATUS Draft v1.0

DATE 2026-09-15

OWNER Adam Helwa

SCOPE Single-family prototype

01 Problem statement

Why this needs to exist, and for whom.

Most families track chores and routines with tools that don't scale past a single whiteboard or sticker chart: they don't travel between two parents, don't adapt to a child's age, and give a kid no feedback loop beyond a checkmark. A 7-year-old and a 15-year-old need entirely different framing for the same underlying habit — "brush your teeth" and "get to practice on time" both deserve to feel motivating, not like a chore log.

At the same time, parents want two things that are usually in tension: **real accountability** (did the task actually happen, not just a self-reported tap) and **low overhead** (they shouldn't have to invent a rewards system or write weekly progress notes by hand).

ChildTaskTracker closes that loop: a child marks a task done, a parent validates it after checking in, and the app handles the tedious part — turning raw completions into a friendly summary, tracking streaks and points, and recommending which reward (from a parent-approved list) actually fits how the child has been doing.

This iteration is scoped as a **single-family prototype** — a mobile-styled front end with a small local backend, built to prove the full loop end-to-end on a near-zero budget, not to ship to an

app store.

02 Personas & journeys

Three people share this app; each sees a different shape of it.

Parent (Co-Manager) PRIMARY USER

Either of two guardians in the same household

Adds children and their tasks, sets the reward catalog and point thresholds, and validates completions after checking in with the child in person. Wants visibility without needing to be the one who logged every task — the other parent's actions should show up just as clearly as their own.

Young Achiever CHILD, 6–9

Needs big, simple, encouraging interactions

Marks tasks done by tapping large icons, sees a mascot, confetti, and streak flames. Doesn't read a KPI dashboard — sees a points jar filling up toward the next reward.

Independent Teen CHILD, 13–16

Wants competence, not cartoons

Same underlying app, quieter visual theme — clean progress bars and a plain streak counter instead of a mascot. Cares more about the allowance/privilege rewards than novelty ones, and doesn't want to feel babied by the interface.

Journey — Setting up a child

1 Add child

Parent enters name and age. Age determines which visual theme and default task suggestions apply.

2 AI suggests starter tasks

On-demand OpenAI call proposes an age-appropriate task list (e.g. "brush teeth," "pack school bag" for a 7-year-old; "study block," "manage own laundry" for a 15-year-old).

3 Parent edits & confirms

Parent accepts, edits, or discards suggestions, then sets each task's recurrence (daily / weekdays / weekly / one-off).

4 Parent builds the reward catalog

Parent adds rewards (movie, ice cream, toy, allowance bump) and sets a point threshold for each — this list is the hard boundary the AI recommender must stay inside.

Journey — A completed task, end to end

1 Child marks a task done

Tap from their own view. Status becomes "Waiting for a parent."

2 Both parents are notified

In-app notification center updates for both Parent A and Parent B (simulated via a view switcher in this prototype), with a toast for visibility.

3 Parent checks in, then validates

A parent talks to the child directly, then taps Approve or Reject in the app — no in-app dispute thread; the conversation happens face to face.

4 Auto-approval fallback

If no parent acts within 24 hours, the task auto-approves so a busy day doesn't cost the child their streak.

5 Points & streak update

Approved completions add points and extend the relevant streak, with one-day forgiveness before a streak resets.

Journey — Redeeming a reward

1 Points cross a threshold

Child's points bank passes a reward's set cost; that reward becomes redeemable in their view.

2 Child requests redemption

Tap "Redeem" — status becomes pending, points are held (not yet spent).

3 Parent approves

Parent confirms in-app; points are deducted and the reward is logged as granted.

AI recommends what's next

On the KPI/rewards screen, parent can request a fresh recommendation — the model picks from the parent's existing reward list based on points balance, streaks, and redemption history, and can suggest adjusting a reward's frequency (e.g. "weekly" vs "one-off").

03 MVP scope

What ships in this prototype versus what's explicitly deferred.

✓ In scope

- Multiple children per family, each with name, age, and age-tier theme
- Task CRUD with mixed recurrence (daily / specific weekdays / weekly / one-off)
- Child-facing task list with one-tap completion
- Parent validation queue, with 24-hour auto-approval
- Points bank + parent-defined reward catalog with per-reward thresholds
- Redemption flow requiring parent approval
- Streaks with one-day forgiveness
- KPIs: completion rate, streaks, points/badges
- OpenAI-powered: starter task suggestions, reward recommendations, KPI summaries, reward-frequency tailoring — all on-demand and constrained to parent-set boundaries
- In-app notification center + toast, visible to both parents via a view switcher
- Age-adaptive theming (playful vs. clean visual skin)

» Deferred

- Real push notifications (FCM/APNs)
- Real multi-device auth / networked co-parent accounts
- In-app dispute or rejection-reason threads
- Category-level KPI breakdowns and historical trend charts
- Fully custom/arbitrary task scheduling
- Extended sharing (grandparents, babysitters, teachers)
- Separated-household (two-account) sharing model
- App store packaging / production deployment

04 Success metrics

Judged two ways: does a real family actually use it, and does the full loop demo cleanly.

Usage metrics (real family use)

METRIC	TARGET	WHY IT MATTERS
Weekly task completion rate	$\geq 70\%$ of assigned tasks	Core signal the habit loop is working, not just installed
Streak retention	≥ 1 active streak / child / week	Shows the forgiveness mechanic is keeping kids engaged, not discouraged
Validation turnaround	$< 24\text{h}$ median	Confirms parents are engaging live, not relying on auto-approval as a crutch
Reward redemption rate	≥ 1 / child / month	Confirms the points system pays off often enough to stay motivating

Technical & demo-completeness metrics

METRIC	TARGET
End-to-end loop demoable	Add task $\rightarrow$ child completes $\rightarrow$ parent validates $\rightarrow$ KPI updates $\rightarrow$ reward recommended, in one sitting
OpenAI cost per active week	$< \$1$
OpenAI call failure handling	Every AI feature has a rule-based fallback if the API errors or times out
Guardrail integrity	100%

05 Technical considerations

A local-first prototype: a real backend and data model, no production hosting.

Stack

Front end	React, mobile-styled layout (max-width shell, safe-area aware), no native build — runs in a browser
Back end	Node.js + Express, running locally

Data store	SQLite (single local file — <code>childtasktracker.db</code>)
AI provider	OpenAI API, called only from the backend (key never reaches the browser)
Auth	None — single running instance; a Parent A / Parent B view switcher simulates two-guardian access

Data model (core entities)

ENTITY	KEY FIELDS
Child	id, name, birthdate, age_tier, avatar
Task	id, child_id, title, recurrence, active
TaskCompletion	id, task_id, marked_at, status (pending/approved/rejected/auto_approved), validated_by
Reward	id, child_id, title, point_cost, category, active
Redemption	id, reward_id, requested_at, status, approved_by
PointsLedger	id, child_id, delta, reason, created_at
Notification	id, recipient (parent_a/parent_b), type, read, created_at

OpenAI integration

Every AI feature is **on-demand** (triggered by a parent action, e.g. tapping "Suggest a reward" or opening the KPI view) and its result is cached until the underlying data changes — never called on every task check-off, to keep this near-zero cost.

Guardrail model: the AI never generates free-form rewards or values. Reward recommendations and frequency tailoring are constrained to select *from* the parent's existing reward catalog and its set point costs — the model ranks and explains, it doesn't invent. KPI summaries and task suggestions use age-tier-appropriate prompt templates (tone/vocabulary) and a fixed system prompt disallowing monetary amounts, screen-time figures, or content outside the parent-approved categories.

- **Starter task suggestions** — fired once, when a child is created, seeded by age.
- **Reward recommendation** — fired on parent request, using points balance, streak state, and redemption history.
- **KPI summary** — fired when a parent opens the KPI view, using that week's completion data.
- **Reward frequency tailoring** — bundled into the recommendation call; suggests whether a reward should be offered more/less often based on how fast the child reaches its threshold.

Every call has a rule-based fallback (e.g. "highest-value reward the child can currently afford") so the app still functions if the API errors, times out, or the key is unset.

06 UI & visual direction

One app, two skins — same structure, different energy.

Young Achiever theme (6–9)

Duolingo-inspired: a mascot companion, bright saturated accents, large tap targets, confetti and streak-flame animation on completion, points shown as a filling jar rather than a number.

Independent Teen theme (13–16)

Same component structure, quieter palette and no mascot — progress bars, a plain streak counter, and reward cards that foreground allowance/privilege-style rewards over novelty ones.

Shared structure

- Mobile-first single-column layout with a bottom tab bar: *Tasks · Rewards · Progress*
- Parent view is a separate mode (Parent A / Parent B switcher) with a validation queue, task/reward editor, and the KPI dashboard
- Notification center as a bell icon with a badge count; tapping opens a chronological list
- Confetti/toast micro-interactions on completion and validation, kept brief and respecting reduced-motion preferences

07 Corner cases

Scenario, and the resolution this PRD commits to.

SCENARIO	RESOLUTION
Parent never validates a completed task	Auto-approves after 24 hours; counts toward points/streak as approved
Parent rejects a task the child marked done	Task returns to "not done," no in-app note — the check-in conversation already happened before the tap
Child misses a day on a streak	One-day forgiveness per streak before it resets to zero

Child tries to redeem with insufficient points	Redeem control stays disabled; shows points remaining to the threshold
Both parents act on the same pending task at once	First write wins; second action shows the already-applied status instead of erroring
A reward is deleted after points were already earmarked toward it	Points remain in the child's general balance, usable toward any other reward
OpenAI API is unreachable or the key is missing	Rule-based fallback runs silently; UI shows the result without surfacing an API error to the child
Parent adds a child with no tasks yet	Empty state prompts "Get starter suggestions" (AI) or "Add your first task" (manual)
Child's age crosses a theme boundary (birthday)	Theme re-evaluates on next login based on current age; no manual migration needed
Two children in the same family, one parent view	A child switcher at the top of the parent view; each child's queue and KPIs are independent

08 Future roadmap

Explicitly out of scope now, worth revisiting if this moves past prototype.

- Real push notifications and networked multi-device access for true two-household use
- Photo-proof validation for tasks where "checking in" isn't practical remotely
- Category-level KPI breakdowns and multi-week trend charts
- Sharing view for extended family (grandparents, babysitters) as read-only
- Separated-household mode with independent parent accounts on a shared child record